

CareBridge – Software Architecture Document v2

CareBridge - Online GP Consultation Platform

Author: Cretu Claudia

Program: Semester 7 Open Learning

1. Introduction

CareBridge is a web platform where patients of a Dutch GP practice submit a structured description of their symptoms, and the practice reviews and answers it without an in-person visit for routine, low-urgency complaints. It is an individual Open Learning project by Cretu Claudia (Fontys ICT, semester 7), built as a learning and portfolio project, not a commercial product.

The platform never suggests or assigns a diagnosis. The patient describes what they experience; only the GP diagnoses and decides whether a live appointment is needed.

1.1 What changed since version 1

Version 1 fixed too many technical choices up front and gave little justification for them. This version keeps only the decisions the project needs now, and records each one with its reasoning in a separate Architecture Decision Record (ADR) document. Everything else is explicitly deferred until a concrete need appears.

Topic	Version 1	Version 2
Database	One SQL database shared by all services	One database per service (ADR-005)
Services	Four services, including a self-built auth service	Three services, login with Google Sign-In (ADR-004, ADR-006)
Service-to-service calls	RabbitMQ for several flows	Plain REST; a broker only when a real need appears (ADR-007)
Deployment	Kubernetes, cloud provider undecided	Early deployment of a walking skeleton with

Topic	Version 1	Version 2
		Docker; Kubernetes deferred (ADR-008)
Diagrams	Four C4 levels	C4 context and container level only
User needs	Not described	Section 2: roles, processes and needs of patients, GPs and GP assistants

1.2 How to read this document

Section 2 describes who uses CareBridge and which practice processes it supports. Section 3 sets the scope for this semester. Sections 4 to 8 describe the architecture at the level of detail that is needed now. Each architectural choice refers to its ADR, where the options and trade-offs are argued.

2. Users and their needs

CareBridge supports the existing triage process of a Dutch GP practice; it does not invent a new one. The needs below come from desk research into how Dutch practices work today, and they are the basis for the scope and the architecture.

2.1 How a Dutch GP practice handles a care question today

In a Dutch practice, the first contact is usually the GP assistant (doktersassistent), not a nurse. The assistant judges the urgency of a care question by phone, at the desk or digitally, and decides whether it must be seen urgently, the same day, later in the week, or whether advice is enough. For this the assistant uses the NHG-TriageWijzer and its protocols; the outcome is an urgency level that says how quickly the GP must assess the patient, not what the follow-up will be.

The GP stays responsible. In a typical practice agreement, the assistant registers every contact in the patient record, the GP checks those contacts, a GP is always available for consultation, and advice follows the NHG standards or is discussed with the GP. Assistants also watch for alarm signals, such as a patient contacting the practice a second time about the same complaint.

2.2 Roles

Version 1 used the role "nurse". This version uses **GP assistant**, because that is the person who triages in a Dutch practice.

Role	What they do in CareBridge	Main needs
Patient	Submits a structured description of a non-urgent complaint and reads the GP's reply	Clear guidance on what the platform is not for (emergencies); a short form; knowing the status of their question; a reply without having to call during busy morning hours
GP assistant	Reviews incoming reports first and puts them in the right order for the GP	Reports sorted so the most serious ones stand out; the ability to override the order with their own judgment; all structured information in one view
GP	Reads a report, writes the reply, and decides whether a live appointment is needed	Enough structured information to decide without calling back; full control over who gets an appointment; nothing that looks like a diagnosis made by software

2.3 Processes CareBridge supports

1. **Submitting a question.** The patient logs in, reads the scope notice (not for urgent or life-threatening situations), and fills in the structured form: complaint area, duration, severity, symptom checklist, treatment already tried, and comments.
2. **Ordering the queue.** New reports are pre-sorted by severity derived from the structured answers, not by an urgency tag the patient picks. The GP assistant reviews the queue and can reorder it manually.
3. **Answering.** The GP reads a report and writes a reply, which the patient receives by email and can also see in the platform.
4. **Escalating to an appointment.** Only when the GP decides a live check-up is needed does the reply contain a single-use appointment link. The booking page is reachable only through that link.

2.4 What these needs mean for the design

- The ordering the platform proposes is a suggestion for the GP assistant, never a medical judgment; the assistant's manual order always wins.

- The platform does not replace the practice's own patient record system (HIS). Connecting to a HIS is out of scope for this semester.
- Health data is sensitive, so access is role-based and every service only sees the data it needs (see ADR-005 and ADR-006).

The needs in this section are based on desk research and are treated as assumptions.

Sources: [practice triage agreement template](#), [Capabel – working as a GP assistant](#), [GP practice patient newsletter Q2 2026](#).

3. Scope for this semester

The semester delivers the four processes from section 2.3 in small increments, starting with a deployed walking skeleton. Features are built in the order below, so each increment can be shown and tested on a running environment.

#	Increment	What it proves
0	Walking skeleton: Angular app and one Spring Boot service with a health endpoint, deployed through the CI pipeline	The build, test and deploy chain works before any feature is built
1	Google Sign-In for patients and practice staff, with staff roles from an allow-list	Role-based access works across front-end and back-end
2	Structured intake form and submission	The Intake Service owns and validates intake data
3	Severity-ordered queue with manual reordering by the GP assistant	The ordering rule and the manual override work together
4	GP reply to the patient by email and in the platform	A second service with its own data, called from the first
5	Appointment link and booking page	Doctor-gated escalation with a single-use token

Out of scope this semester: connecting to a practice's patient record system (HIS), video consultation, prescriptions, a mobile app, and any form of automated diagnosis and real stakeholder research. Load testing and container orchestration are deferred until the core increments run (see section 4.2).

4. Architecture overview

CareBridge is an Angular web app in front of three Spring Boot services, each with its own database, with login handled by Google Sign-In. Only decisions that block the first increments are taken now; the rest wait until there is a concrete need and real information to base them on.

4.1 Guiding principles

- **Patient safety first.** No feature suggests a diagnosis. The GP decides; the GP assistant can always override what the software proposes.
- **Decide at the last responsible moment.** A decision is made when an increment needs it, and every decision is recorded as an ADR with the options considered.
- **A service owns its data.** No service reads or writes another service's database. Data is shared only through an API.
- **Deploy early, deploy often.** The platform runs in a real environment from increment 0, so problems with configuration and infrastructure appear early.

4.2 Decisions made now

ADR	Decision
ADR-001	Record architecture decisions as ADRs
ADR-002	Angular single-page application as the one front-end for all roles
ADR-003	Java with Spring Boot for the back-end services
ADR-004	Three services split by business capability: Intake, Messaging, Appointment
ADR-005	One PostgreSQL database per service
ADR-006	Login with Google Sign-In (OpenID Connect)
ADR-007	Synchronous REST between services; no message broker for now

ADR	Decision
ADR-008	Early deployment of Docker containers to a managed container platform
ADR-009	Two repositories: front-end, and back-end with an independent build and pipeline per service

4.3 Decisions deliberately deferred

Topic	Decide when	Why not now
Message broker (e.g. RabbitMQ)	A flow must survive another service being down, or one event has several consumers	No flow in increments 0–5 needs it yet
Container orchestration (Kubernetes)	The managed container platform limits scaling or roll-out	Three small services run fine without it; it adds a lot of setup for no current benefit
API gateway	The front-end needs one entry point for CORS, routing or rate limiting	With three services, direct calls are simpler to build and debug
Email provider	Increment 4	The choice depends on the hosting platform and free tier at that moment
Live queue updates (polling or WebSockets)	Increment 3, after the GP assistant view exists	Polling may be enough; measure first
Load and performance testing tool	After increment 5 runs in the cloud	Results are only meaningful on the deployed environment

5. Service boundaries and data ownership

Each service matches one step of the practice process in section 2.3 and owns the data for that step in its own database. Services refer to each other's data only by ID, never by reading another database.

Service	Responsibility	Owns (own database)	Main endpoints
Intake Service	Receives and validates submissions; builds the severity-ordered queue; stores the GP assistant's manual order; tracks the status of each intake	Intakes and their structured answers, queue position, status	POST /intakes, GET /intakes/{id}, GET /queue, PUT /queue/order, PATCH /intakes/{id}/status
Messaging Service	Stores the GP's reply, sends it by email, shows it to the patient in the platform	Replies and the email delivery log	POST /replies, GET /replies?intakeId=
Appointment Service	Creates single-use appointment links, validates them, and stores bookings	Link tokens, available slots, bookings	POST /appointment-links, GET /appointment-links/{token}, POST /bookings
Google Sign-In (external)	Login and identity (user ID, email); staff roles come from an allow-list in the services' configuration	Google accounts and credentials	Standard OpenID Connect endpoints

5.1 Why these boundaries

- **They change for different reasons.** The intake form and ordering rules change with triage practice; the reply and email flow changes with the email provider; booking changes with how the practice plans its hours.
- **They fail differently.** An email provider outage must not stop patients from submitting questions. With separate services, the Intake Service keeps working when the Messaging Service or its provider is down.
- **They carry different load.** Intake submissions peak when practices open in the morning; appointment booking is rare. Separate services can be scaled on their own.
- **Login is not built in-house.** Account security is a solved problem; an external provider removes the most security-sensitive code from the project (ADR-006).

5.2 Data shared between services

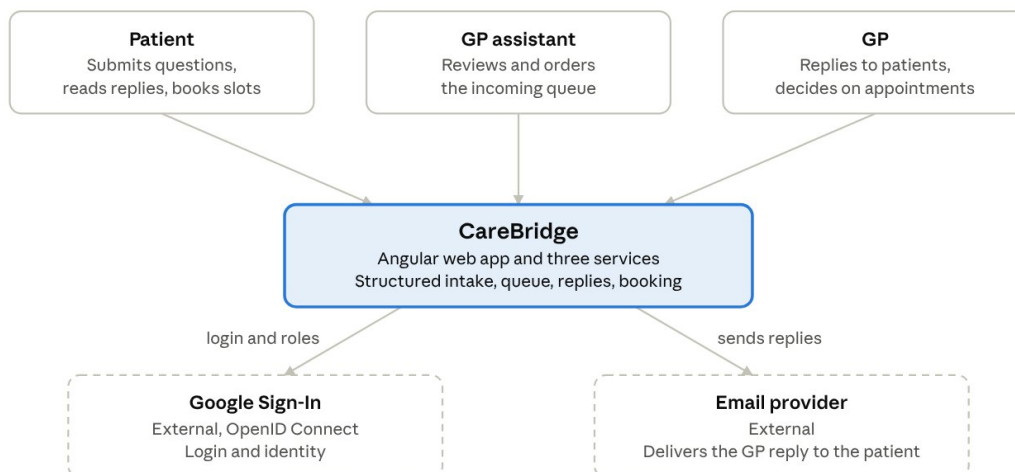
Data	Owner	Used by	How
User ID and role	Google Sign-In (ID); service configuration (role)	All services	Read from the signed access token on every request
Patient email address	Google Sign-In	Messaging Service	Claim in the token of the patient, stored with the intake at submission
Intake ID	Intake Service	Messaging, Appointment	Passed as a reference in API calls
"Answered" status	Intake Service	Messaging Service	Messaging calls <code>PATCH /intakes/{id}/status</code> after a reply is sent

6. Architecture diagrams

Two C4 levels are enough for the current stage: the context shows who uses CareBridge, and the container view shows how it is built. Component and code diagrams are added per service once that service's code exists.

6.1 System context (C4 level 1)

Patients and practice staff use CareBridge; login and email are external



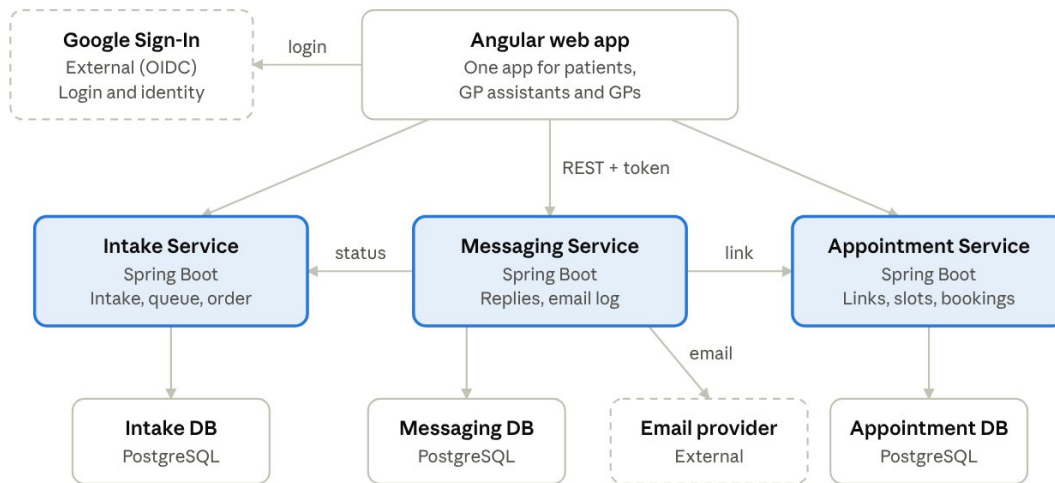
Dashed boxes are external systems.

C4 level 1 · system context

Three roles use one platform. CareBridge depends on two external systems: one for login and one for sending email.

6.2 Containers (C4 level 2)

Each service owns its database; only Messaging calls other services



C4 level 2 · containers

The Angular app calls each service directly over REST. The Messaging Service is the only service that calls others: it asks the Appointment Service for a link and tells the Intake Service that a question is answered.

7. Key flows

Three flows cover the whole patient journey; each involves at most two services, which keeps failures easy to trace.

7.1 Submitting a question

1. The patient signs in with Google and the app receives a Google ID token.
2. The Angular app shows the scope notice; the patient confirms it and fills in the structured form.
3. The app sends the form to `POST /intakes`. The Intake Service validates it, calculates a severity score from the structured answers, and stores it.
4. The patient sees the status "Submitted". No email is sent at this stage.

7.2 Ordering and answering

1. The GP assistant opens the queue (GET /queue), sorted by severity score. They drag reports into a different order if their judgment differs; the app saves it with PUT /queue/order.
2. The GP opens a report and writes a reply. If a live check-up is needed, they tick "include appointment link".
3. The app sends the reply to POST /replies. If a link is requested, the Messaging Service first calls POST /appointment-links on the Appointment Service and adds the returned link to the email.
4. The Messaging Service sends the email, stores the reply, and sets the intake to "Answered" through the Intake Service.

If the Appointment Service or the email provider is unavailable, the reply is not sent and the GP sees an error with the option to retry. The Intake Service and the queue keep working. This is the first flow to move to asynchronous messaging if retries become a problem (see ADR-007).

7.3 Booking an appointment

1. The patient opens the link from the email. The Angular route guard sends the token to GET /appointment-links/{token}.
2. The Appointment Service checks that the token exists, has not expired, and has not been used. Only then does the booking page show free slots.
3. The patient picks a slot; POST /bookings stores it and marks the token as used.

8. Deployment plan

The walking skeleton (increment 0) is deployed to the cloud before any feature is built, and every later increment goes through the same pipeline. This surfaces configuration, networking and secret-handling problems while the system is still small.

8.1 Environments

Environment	Runs on	Purpose
Local	Docker Compose on the developer laptop: Angular app, three services, PostgreSQL, Google Sign-In through a test OAuth client	Fast development and debugging
Cloud (test)	Azure Container Apps for the containers, one managed PostgreSQL server with a separate database and user	Show working increments and test in a realistic setting

Environment	Runs on	Purpose
	per service	

Each service connects with its own database user, which only has rights on its own database. This keeps data ownership strict even though the three databases share one server to stay within the student budget (ADR-005, ADR-008).

8.2 Pipeline

1. **On every push:** GitHub Actions builds and unit-tests only the parts that changed: the Angular app in the front-end repository, or the individual services whose folder changed in the back-end repository (JUnit for services, Karma/Jasmine for the Angular app). A failing test stops the pipeline.
2. **On merge to main:** the pipeline builds a Docker image per changed service, tags it with the commit hash, and pushes it to GitHub Container Registry.
3. **Deploy:** the pipeline updates the matching container app in Azure. Secrets such as database passwords are stored in GitHub and Azure secret stores, never in the repository.

8.3 First deployment checklist (increment 0)

- ☐ Angular app served from a container, reachable over HTTPS
- ☐ Intake Service with a /health endpoint, connected to its own database
- ☐ Pipeline runs build, test, image push and deploy on merge to main
- ☐ Secrets configured outside the repository

9. Open questions and next steps

The next step is increment 0: the deployed walking skeleton, followed by the identity-provider spike that settles ADR-006.

- ☐ Google Sign-In: confirm in the increment 1 spike that sign-in, token validation and the staff allow-list work locally and in the cloud.
- ☐ Severity score: which structured answers weigh most, and should the GP assistant also set an urgency level next to reordering? Decided in increment 3.
- ☐ How long are appointment links valid (for example 48 hours or 7 days)? Decided in increment 5.
- ☐ Does polling every few seconds give the GP assistant a fresh enough queue, or are WebSockets needed? Measured in increment 3.